ESGI · ASSEMBLEUR X86

Fiche réflexe — Assembleur x86 32 bits

À garder à côté de soi pendant les TP

Enseignant : **Youssef YOUNES**

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

Les registres

Des cases de 4 octets (32 bits) dans le processeur. On y calcule.

Registre	Usage dans nos TP
`eax`	le résultat de la fonction (main.c le lit ici)
`ecx`	le compteur de boucle
`edx`	une deuxième valeur à comparer, un temporaire
`esi`	une adresse pour parcourir un tableau
`ebx` , `edi` , `ebp` , `esp`	ne pas toucher pour l'instant

Crochets ou pas ?

Écriture	Signification
`mov eax, 10`	eax = le nombre 10
`mov eax, [a]`	eax = la valeur de la variable a
`mov esi, tableau`	esi = l'adresse du tableau
`add eax, [esi]`	eax += le contenu à l'adresse esi
`add esi, 4`	case suivante (un entier = 4 octets)

Règle : un seul opérande en mémoire par instruction. `add [a], [b]` est interdit : passer par un registre.

Squelette d'une fonction

```
section .data
global a
a:  dd 0            ; variable (4 octets)

section .text
global ma_fonction
ma_fonction:
    mov  eax, [a]   ; ... calculs ...
    ret             ; résultat : eax
```

Une étiquette = un nom + deux-points, seul sur sa ligne. Un commentaire commence par `;` .

Les instructions du TP 1

`dst` = destination (à gauche), `src` = source (à droite).

Instruction	Effet
`mov dst, src`	dst = src (copie)
`add dst, src`	dst = dst + src
`sub dst, src`	dst = dst - src
`imul dst, src`	dst = dst × src (signé)
`inc dst` / `dec dst`	dst = dst + 1 / dst - 1
`neg dst`	dst = -dst
`cmp a, b`	compare a et b, sans les modifier (toujours suivi d'un saut)
`jmp etiq`	saute toujours
`je` / `jne etiq`	saute si égal / différent
`jg` / `jge etiq`	saute si a > b / a ≥ b (signé)
`jl` / `jle etiq`	saute si a < b / a ≤ b (signé)
`ret`	fin de la fonction

Les deux schémas à connaître

Si … alors (le saut enjambe le bloc) :

```
    cmp  eax, edx
    jge  fin            ; eax >= edx : saute
    mov  eax, edx       ; si eax < edx
fin:
```

Tant que compteur ≠ 0 :

```
    mov  ecx, [n]
boucle:
    cmp  ecx, 0
    je   fin            ; sortie
    ; ... travail ...
    dec  ecx
    jmp  boucle         ; retour en haut
fin:
```

Commandes : `make test` vérifie · `make run` exécute · `make trace` affiche les registres après chaque instruction (`HEX=1` : hexadécimal, `IN=tests/x.in` : autre entrée).